

RAPPORT DE PROJET

Conception d'Algorithmes

Résolution du Problème du Sac à dos 0/1

Comparaison de l'efficacité de différentes méthodes en termes de complexité temporelle et spatiale.

Réalisé par : Mathura SANTHALINGAM

Encadrant : M. Tomeh

Question 1 :

Le format utilisé dans le fichier est le csv (comma-separated values) : dans ce format la séparation des champs se fait par des virgules, les informations sont donc stockées entre celles-ci.

Notons que la première case de chaque ligne représente la capacité maximale du sac à dos tandis que la deuxième case, elle, représente le nombre d'items. Dans le reste des cases, on retrouve des couples formant le poids et la valeurs associés à chaque item.

Par exemple, une ligne du fichier ressemblerait à ceci : 175, 5, 7, 2, 10, 2, 7, 8, 3, 2, 1, 8.
Ici, **W = 175**, **n = 5**, les objets ont des poids et valeurs (7, 2), (10, 2), (7, 8), (3, 2), et (1, 8).

Remarque : On note ainsi que l'on effectue 10 tests pour un nombre d'objets compris entre 5 et 25. (NUM_INSTANCE_PER_SIZE = 10)

```
1
2 //INSTANCES.CSV
3
4 175,5,7,2,10,2,7,8,3,2,1,8
5 199,5,7,8,4,9,7,5,4,1,7,7
6 177,5,2,10,6,6,1,2,10,4,3,1
7 140,5,10,7,7,8,4,6,2,2,8,6
8 159,5,3,3,4,7,5,10,9,4,5,9
9 189,5,2,8,7,6,7,4,1,10,7,10
10 150,5,6,4,8,5,8,4,8,1,6,7
11 172,5,3,10,7,6,6,4,4,6,8,4
12 105,5,6,2,1,2,10,1,3,7,10,2
13 133,5,3,5,3,2,1,6,10,3,9,4
14 105,6,1,2,4,6,7,2,4,6,5,6,3,8
15 157,6,8,7,3,2,3,9,8,1,4,8,3,2
16 169,6,7,8,3,4,6,4,5,5,5,8,5,7
17 178,6,7,7,3,4,1,4,7,10,8,1,2,3
18 150,6,5,7,8,9,10,10,3,2,8,9,7,6
19 199,6,1,7,8,7,1,5,6,1,4,3,1,3
20 106,6,8,7,6,9,9,2,2,7,6,5,4,7
21 198,6,4,10,9,10,3,7,10,2,8,10,7,10
22 103,6,4,4,5,9,5,10,9,5,9,1,6,8
23 121,6,1,6,6,5,4,4,8,7,7,1,7,10
24 105,7,7,4,4,6,5,8,8,5,9,3,10,4,8,3
25 133,7,4,7,1,1,1,4,10,2,6,6,4,5,6,5
26 162,7,1,7,7,9,5,8,9,1,10,8,10,10,6,10
27 152,7,2,3,6,7,8,3,8,8,8,6,9,6,7,5
28 101,7,9,9,1,7,1,8,1,8,5,9,1,4,9,3
29 165,7,4,9,4,1,7,2,3,9,8,3,5,10,6,3
30 171,7,2,8,7,10,6,6,2,9,1,5,2,3,9,6
```

Figure 1 : instances.csv

Question 2 :

Complexité temporelle :

`knapsackBT1` et `knapsackBT2` ont la même complexité : puisque l'algorithme doit décider d'inclure ou non cet objet, on a un nombre exponentiel de combinaisons à examiner il a donc deux appels récursifs par itération. Les deux ont ainsi une complexité de $O(2^n)$.

Utilisation en mémoire :

La différence se fait en terme d'utilisation de la mémoire.

`knapsackBT1` : utilise des variables globales, donc la pile récursive est relativement plus petite.

`knapsackBT2` : consomme plus de mémoire pour chaque appel récursif en raison des paramètres supplémentaires passés à chaque niveau de la pile d'appels.

Ainsi, les deux versions nécessitent une profondeur de pile maximale de n , ce qui signifie que la consommation de mémoire en termes de pile d'exécution est également $O(n)$.

Variables Globales :

La version avec variables locales consomme plus de mémoire pour chaque appel récursif en raison des paramètres supplémentaires passés à chaque niveau de la pile d'appels. Cependant, elle est plus encapsulée et évite les problèmes potentiels liés aux variables globales, comme les effets de bord non intentionnels et les difficultés de débogage.

En résumé, la version avec variables globales utilise moins de mémoire par niveau de récursion, mais au prix d'une moindre encapsulation et d'une potentialité accrue pour des bugs difficiles à tracer. La version avec variables locales utilise plus de mémoire mais offre un code plus propre et mieux structuré.

Question 3 :

L'algorithme de backtracking commence à devenir impraticable pour des valeurs de n supérieures à 25.

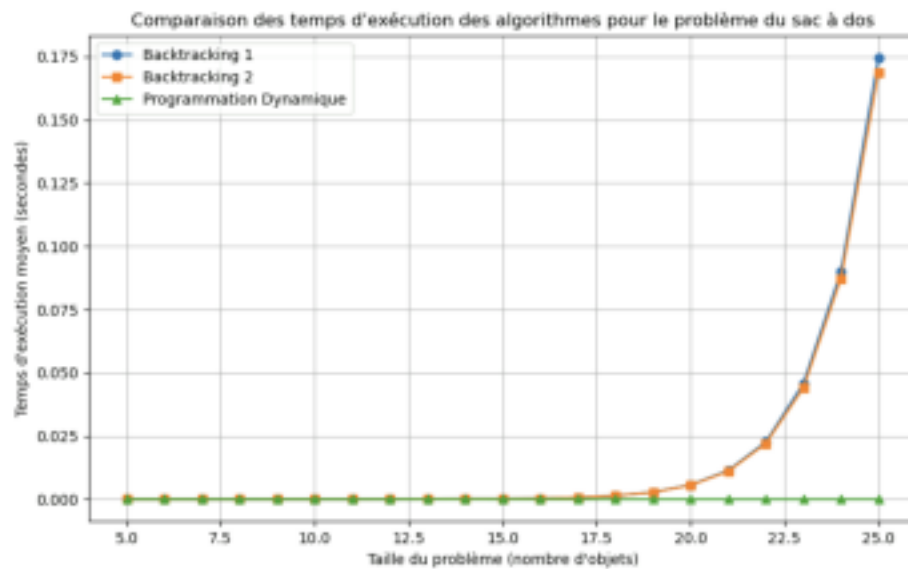


Figure 2 : Temps d'exécution (en seconde) pour $n = 25$

Remarque : on a une latence maximale de 0.175 secondes.

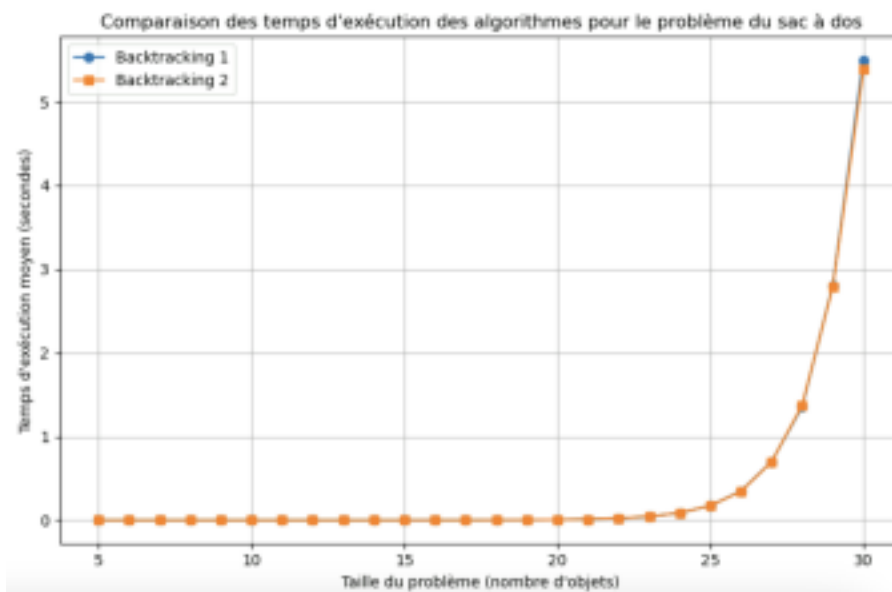


Figure 3 : Temps d'exécution (en seconde) pour $n = 30$

Remarque : on a une latence maximale de 5 secondes ce qui est beaucoup trop lent.

Question 4 :

On précalcule la borne sup (bs) et on appelle la fonction knapsackBTUtilbs

Fonction ajoutée :

```
int knapxsackBT2bs(Item items[], int n, int W) {
    int bestValue = 0;
    int i;
    int bs[n];
    bs[n-1] = items[n-1].value; //bs quand il reste que le dernier elem

    //précalcul
    for(i = n-2; i >= 0 ; i-- )
    {
        bs[i] = bs[i+1] + items[i].value;
    }

    /* Autre manière :
    for(i = 2; i < n+1 ; i++ )
    {
        bs[n-i] = bs[n-i+1] + item[n-i].value;
    }*/

    return knapsackBTUtilbs(items, n, W, 0, 0, &bestValue, bs);
}
```

// Algorithme de backtracking sans variables globales **AVEC BORNE SUP**

```
int knapsackBTUtilbs(Item items[], int n, int W, int idx, int
currentValue, int *bestValue, int * bs) {
    if (idx == n || W == 0)
    {
        if (currentValue > *bestValue)
            *bestValue = currentValue;
        return *bestValue;
    }

    if( bs[idx] + currentValue < *bestValue )
        return *bestValue;

    // Ne pas inclure l'objet idx

    knapsackBTUtilbs(items, n, W, idx + 1, currentValue,bestValue,
    bs); // Inclure l'objet idx si le poids le permet

    if (items[idx].weight <= W)
    {
        knapsackBTUtilbs(items, n, W - items[idx].weight, idx + 1,
currentValue + items[idx].value, bestValue, bs);
    }

    return *bestValue;
}
```

Efficacité :

L'algorithme n'est pas toujours plus efficace. Par exemple, si le poids de l'ensemble de tous les éléments est inférieur à la capacité maximale, alors il ne sert à rien d'envisager toutes les possibilités d'élagage. Cependant, il marche mieux quand les éléments sont triés selon l'ordre décroissant de leur valeur. Ainsi, en moyenne, il reste plus efficace que les backtracking vus précédemment.

```
3 //AVERAGE-TIMES.CSV
4
5 5, 0.000001, 0.000002, 0.000002
6 6, 0.000002, 0.000002, 0.000002
7 7, 0.000004, 0.000004, 0.000002
8 8, 0.000006, 0.000007, 0.000003
9 9, 0.000011, 0.000011, 0.000004
10 10, 0.000020, 0.000023, 0.000005
11 11, 0.000043, 0.000044, 0.000006
12 12, 0.000034, 0.000035, 0.000002
13 13, 0.000043, 0.000044, 0.000002
14 14, 0.000086, 0.000088, 0.000003
15 15, 0.000170, 0.000173, 0.000003
16 16, 0.000337, 0.000342, 0.000003
17 17, 0.000685, 0.000688, 0.000005
18 18, 0.001345, 0.001365, 0.000005
19 19, 0.002716, 0.002751, 0.000006
20 20, 0.005498, 0.005528, 0.000007
21 21, 0.010827, 0.010999, 0.000009
22 22, 0.021526, 0.022065, 0.000010
23 23, 0.043571, 0.044126, 0.000012
24 24, 0.084534, 0.086562, 0.000033
25 25, 0.173306, 0.174264, 0.000020
```

Figure 4 : average_times.csv de BT1(1ère colonne), BT2 (2ème) et BT2bs(3ème)



Figure 5 : Temps d'exécution (en seconde) pour $n = 25$

Question 5 :

Sous structure optimale:

Comme vu lors du cours magistral, le problème du sac à dos a la propriété de sous structure optimale. En effet, on a bien que si $O_{i1}, O_{i2}, \dots, O_{ik}$ est une solution optimale pour un sac à dos de capacité maximale W avec une valeur totale p et un poids total w , alors $O_{i1}, O_{i2}, \dots, O_{ik-1}$ est une solution optimale pour un sac à dos de capacité maximale $W - w_{ik}$. Pour démontrer cela par l'absurde, supposons qu'il existe une meilleure solution pour $W - w_{ik}$ avec une valeur $p' > p - p_{ik}$. En ajoutant O_{ik} à cette solution, nous obtenons une valeur $p' + p_{ik}$, qui serait supérieure à p , ce qui contredit l'hypothèse que p soit la valeur optimale. Par conséquent, $O_{i1}, O_{i2}, \dots, O_{ik-1}$ doit être une solution optimale pour $W - w_{ik}$.

Définition récursive pour $dp[v]$:

$$dp[v] = \min(dp[v], w_i + dp[v - v_i])$$

Avec :

- $dp[v]$ qui représente le poids minimal nécessaire pour atteindre une valeur v .
- w_i est le poids de l'objet i .
- v_i est la valeur de l'objet i .

Question 6 :

Fonction ajoutée :

```
int knapsackDP_Value(Item items[], int n, int W ) {  
  
    int V = 0;  
  
    for(int j= 0; j < n ; j++)  
        { V+=items[j].value; }  
  
    int dp[V+1];  
    int k = V;  
  
    while(k > 0)  
        {dp[k] = INT_MAX;  
         k--;}  
  
    dp[0]= 0;  
  
    for(int i = 0 ; i < n ; i++ ){  
        for(int v = V ; v >= items[i].value ; v--)  
            {  
                if( dp[v - items[i].value] != INT_MAX )  
                    dp[v]=min( dp[v] , dp[ v - items[i].value ] +  
                        items[i].weight); }  
    }  
    for( int i = V; i>0 ; i--)  
        {if( dp[i] <= W) return i;}  
  
    return 0;  
}
```

Exemple : Pour $V = 100$

Si l'Objet 0 a pour valeur 10 et poids 5, on remplit la case $dp[10]=5$

Puis, si l'Objet 1 a pour valeur 20 et poids 4, on remplit la case $dp[20] = 4$ et $dp[30] = 9$

De même, si l'Objet 2 a pour valeur 50 et poids 3, on remplit la case $dp[50] = 3$, $dp[60] = 8$, $dp[70] = 7$ et $dp[80] = 12$.

Complexité temporelle de l'algorithme knapsackDP_Value

La complexité temporelle de l'algorithme `knapsackDP_Value` est déterminée par les boucles imbriquées dans la fonction, (car les autres boucles sont négligeables). Dans le pire des cas, la boucle interne est parcourue V fois, et celle externe n fois. Ce qui donne une complexité de $O(nV)$.

Modifications de la fonction main()

```
int main() {
    /* On se permet de ne mentionner que les changements apportés */

    double sumTimeBT1 = 0, sumTimeBT2 = 0, sumTimeDP = 0, sumTimeBT2bs =
    0, sumTimeDPV = 0;
    int count = 0, currentN = -1;
    /* [...] */
    if (n != currentN)
    {
        if (count > 0)
        {
            fprintf(out, "%d, %f, %f, %f, %f, %f\n", currentN, sumTimeBT1 /
count, sumTimeBT2 / count, sumTimeDP / count, sumTimeBT2bs /count,
sumTimeDPV /count);
        }
        currentN = n;
        sumTimeBT1 = sumTimeBT2 = sumTimeDP = sumTimeBT2bs = sumTimeDPV =
        0; count = 0;
    }
    /* [...] */
    start = clock();
    int maxValBT2bs = knapsackBT2bs(items, n, W);
    end = clock();
    double timeBT2bs = (double)(end - start) /
CLOCKS_PER_SEC; sumTimeBT2bs += timeBT2bs;

    start = clock();
    int maxValDPV = knapsackDP_Value(items, n, W);
    end = clock();
    double timeDPV = (double)(end - start) /
CLOCKS_PER_SEC; sumTimeDPV += timeDPV;
    count++;
    fprintf(sol, "%d, %d, %f, %d, %f, %d, %f, %d, %f, %d, %f, %d\n", n, W,
timeBT1, maxValBT1, timeBT2, maxValBT2, timeDP, maxValDP, timeBT2bs,
maxValBT2bs, timeDPV, maxValDPV);
}

    if (count > 0) {
        fprintf(out, "%d, %f, %f, %f, %f, %f\n", currentN, sumTimeBT1
/ count, sumTimeBT2 / count, sumTimeDP / count, sumTimeBT2bs /count,
sumTimeDPV /count);
    }
    /* [...] */
}
```

Efficacité Par rapport à DP :

```
3 //AVERAGE_TIMES.CSV
4
5 5, 0.000016, 0.000003
6 6, 0.000018, 0.000004
7 7, 0.000018, 0.000005
8 8, 0.000019, 0.000006
9 9, 0.000024, 0.000006
10 10, 0.000028, 0.000008
11 11, 0.000027, 0.000008
12 12, 0.000014, 0.000004
13 13, 0.000009, 0.000003
14 14, 0.000010, 0.000003
15 15, 0.000010, 0.000003
16 16, 0.000008, 0.000004
17 17, 0.000011, 0.000004
18 18, 0.000012, 0.000005
19 19, 0.000013, 0.000005
20 20, 0.000016, 0.000007
21 21, 0.000017, 0.000007
22 22, 0.000018, 0.000008
23 23, 0.000019, 0.000009
24 24, 0.000016, 0.000009
25 25, 0.000017, 0.000010
```

Figure 6 : average_times.csv de DP (1ère colonne) et DPV (2ème colonne)

Modifications apportées au fichier plot_temps.py

```
#On se permet de ne mentionner que les changements
apportés #[...]
df = pd.read_csv(csv_file_path, header=None, names=['Taille',
'BT1', 'BT2', 'BT2bs', 'DP', 'DPV'])

# Préparation du graphique pour comparer les temps d'exécution des
trois algorithmes
plt.figure(figsize=(10, 6))
#plt.plot(df['Taille'], df['BT1'], label='Backtracking 1',
marker='o') #plt.plot(df['Taille'], df['BT2'], label='Backtracking
2', marker='s') #plt.plot(df['Taille'], df['BT2bs'],
label='Backtracking BS 2', marker='s')
plt.plot(df['Taille'], df['DP'], label='Programmation
Dynamique', marker='^')
plt.plot(df['Taille'], df['DPV'], label='Programmation Dynamique
Value', marker='^')
#[...]
```

Courbes :

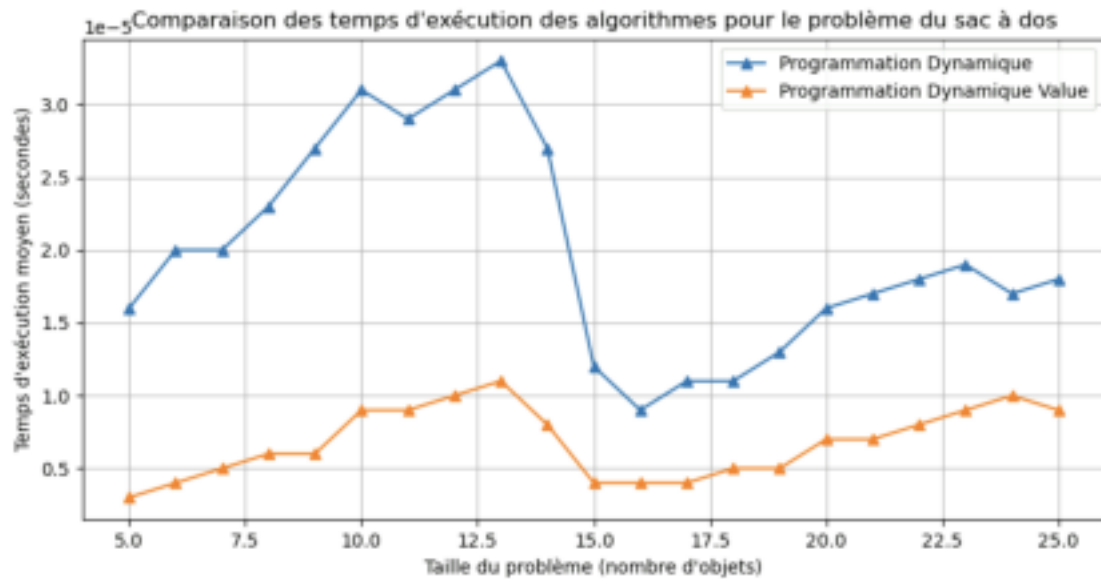


Figure 7 : Temps d'exécution (en sec) pour $n=25$ et $\text{NUM_INSTANCE_PER_SIZE} = 10$

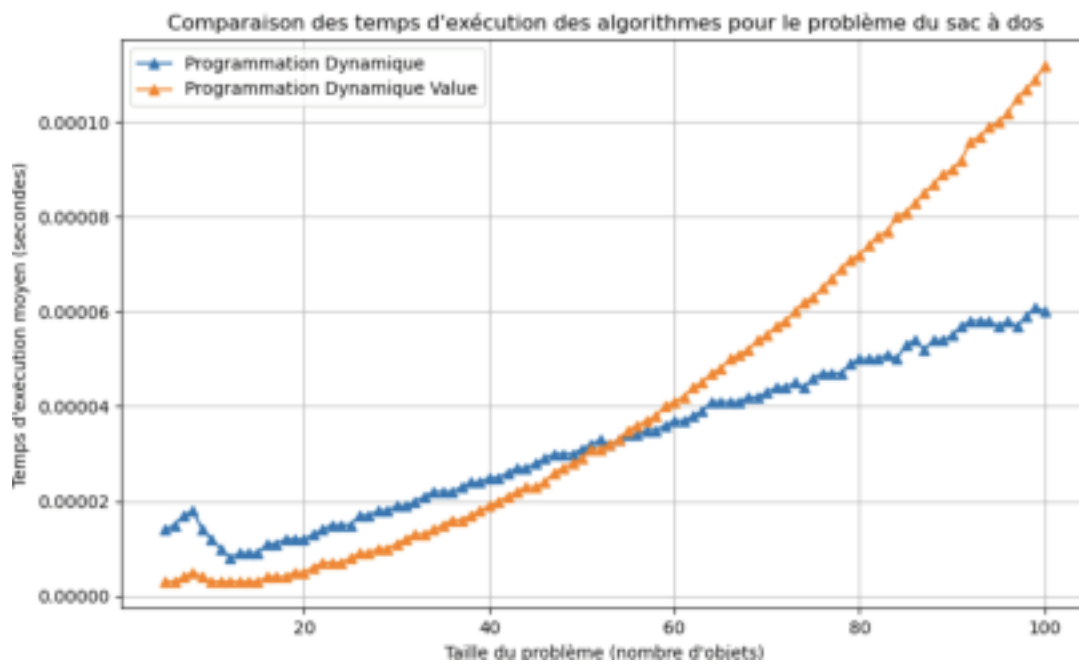


Figure 8 : Temps d'exécution (en sec) pour $n=100$ et $\text{NUM_INSTANCE_PER_SIZE} = 100$

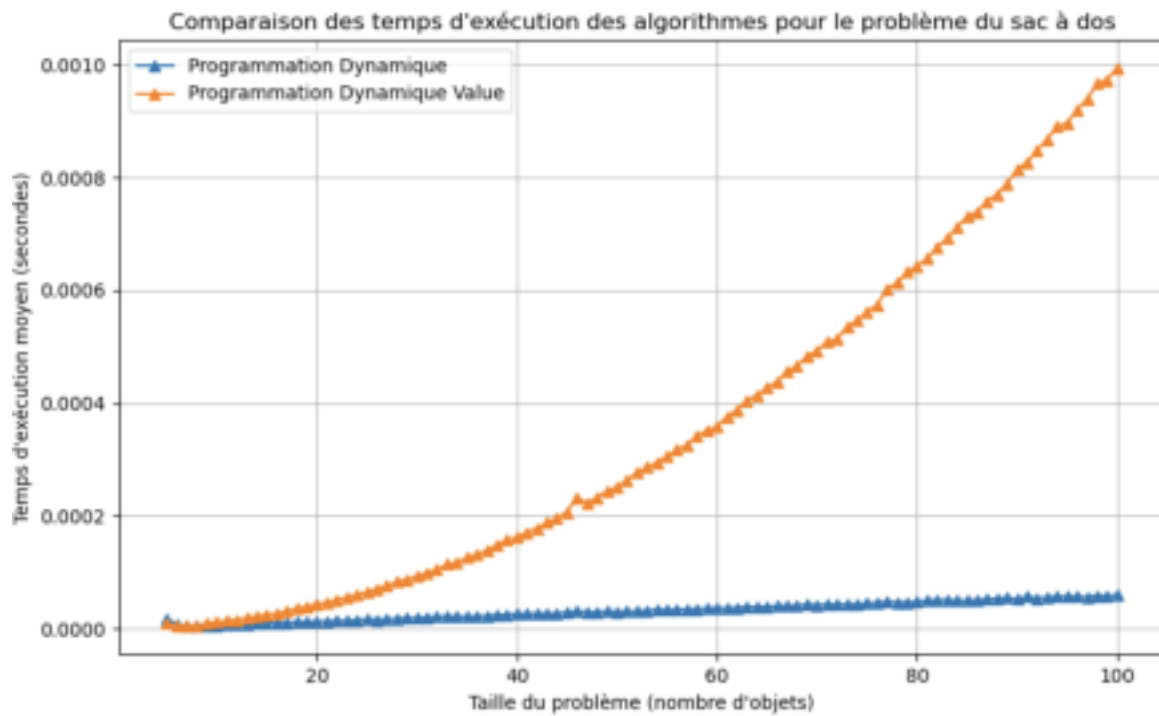


Figure 9 : Temps d'exécution (en sec) pour $n = 100$, $\text{NUM_INSTANCE_PER_SIZE} = 100$ et $\text{MAX_WEIGHT} = 100$

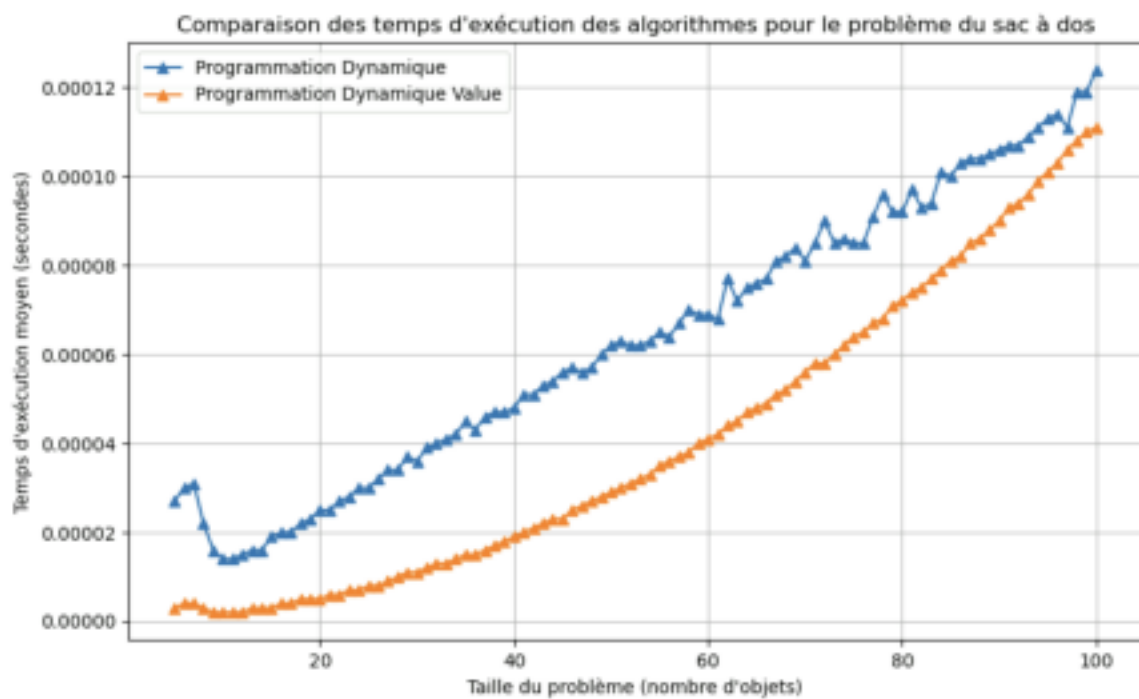


Figure 10 : Temps d'exécution (en sec) pour $n = 100$, $\text{NUM_INSTANCE_PER_SIZE} = 100$ et $\text{MAX_CAPACITY} = 400$

Question 7 :

On propose de stocker dans $dp[i][w]$ la valeur maximale que l'on puisse atteindre en ayant considéré les i premiers objets dont le poids totale est exactement w .

Définition récursive

$$dp[i][w] = \max(dp[i-1][w], v_i + dp[i-1][w - w_i])$$

Fonction ajoutée

```
int knapsackDP_Weight(Item items[], int n, int C, int M) {
    int dp[n + 1][C + 1];
    fprintf(stderr, "C = %d, M = %d, n = %d item[0].value = %d, items[1].value = %d \n", C, M, n, items[0].value, items[1].value);
    for(int w = 0; w <= C ; w++){
        dp[0][w] = 0;
        fprintf(stderr, "dp[0][w] = %d\n", dp[0][w]);
    }
    for(int w = 0; w <= n ; w++){
        dp[w][0] = 0;
        fprintf(stderr, "dp[w][0] = %d\n", dp[w][0]);
    }
    for(int i = 1; i <= n ; ++i){
        for(int w = 1 ; w <= C ; ++w){

            if (items[i-1].weight <= C){
                if(w < items[i-1].weight){
                    dp[i][w] = dp[i-1][w];
                    printf("1 dp[%d][%d] = %d\n",i, w ,dp[i][w]);}
                else if(w == items[i-1].weight){
                    dp[i][w] = max(dp[i-1][w], items[i-1].value);
                    printf("2 dp[%d][%d] = %d\n",i, w, dp[i][w]);}
                else if(dp[i-1][w-items[i-1].weight] != 0){
                    dp[i][w] = max(dp[i-1][w],
                    dp[i-1][w-items[i-1].weight]+ items[i-1].value);
                    printf("3 dp[%d][%d] = %d\n",i, w, dp[i][w]);}
                else{
                    dp[i][w] = dp[i-1][w];
                    printf("4 dp[%d][%d] = %d\n",i, w ,dp[i][w]);}
            }
            else {dp[i][w] = dp[i-1][w];
            printf("5 dp[%d][%d] = %d\n",i, w, dp[i][w]);}
        }
    }

    int max_w = 0;

    for(int m = M; m <= C ; m++){
        {
            if(dp[n][m] > max_w){
                max_w = dp[n][m];
            }
            // max(max_w, dp[n][m]);
        }
        fprintf(stderr, "max_w = %d \n", max_w);
        return max_w;
    }
}
```

Modifications apportées dans la fonction main()

```
/* On se permet de ne mentionner que les changements apportés
*/ /* [...] */
double sumTimeBT1 = 0, sumTimeBT2 = 0, sumTimeDP = 0, sumTimeBT2bs =
0, sumTimeDPV = 0, sumTimeDPW = 0;
int count = 0, currentN = -1;
char line[4096];
while (fgets(line, 4096, fp))
{
    int W, n, M;
    sscanf(line, "%d,%d,%d", &W, &n, &M;);
    if (n != currentN)
    {
        if (count > 0)
        {
            fprintf(out, "%d, %f, %f, %f, %f, %f, %f\n", currentN, sumTimeBT1
/ count, sumTimeBT2 / count, sumTimeDP / count, sumTimeBT2bs /count,
sumTimeDPV / count, sumTimeDPW / count;)
        }
        currentN = n;
        sumTimeBT1 = sumTimeBT2 = sumTimeDP = sumTimeBT2bs = sumTimeDPV
= sumTimeDPW = 0;
        /* [...] */
        token = strtok(NULL, ",");
        /* [...] */
        start = clock();
        int maxValDPW = knapsackDP_Weight(items, n, W, M); end =
clock(); double timeDPW = (double)(end - start) /
CLOCKS_PER_SEC;
        sumTimeDPW += timeDPW; count++;
        fprintf(sol, "%d, %d, %f, %d, %f, %d, %f, %d, %f, %d, %f, %d,
%f, %d\n", n, W, timeBT1, maxValBT1, timeBT2, maxValBT2, timeDP,
maxValDP, timeBT2bs, maxValBT2bs, timeDPV, maxValDPV, timeDPW,
maxValDPW;)
    }
    if (count > 0)
    {
        fprintf(out, "%d, %f, %f, %f, %f, %f, %f\n", currentN, sumTimeBT1
/ count, sumTimeBT2 / count, sumTimeDP / count, sumTimeBT2bs /count,
sumTimeDPV /count, sumTimeDPW / count;)
    }
    /* [...] */
}
```

Modifications apportées dans generateData.py

```
# [...]  
MIN_CAPACITY = 50  
# [...]  
for num_items in range(MIN_ITEMS, MAX_ITEMS + 1,  
    ITEM_STEP): for _ in range(NUM_INSTANCES_PER_SIZE) :  
    capacity = random.randint(MAX_CAPACITY // 2,  
        MAX_CAPACITY) mincap = random.randint(MIN_CAPACITY,  
        MIN_CAPACITY * 2)  
    weights = [random.randint(1, MAX_WEIGHT) for _ in  
range(num_items)] values = [random.randint(1, MAX_VALUE) for _ in  
range(num_items)] instance = [capacity, num_items, mincap] + [val for  
pair in zip(weights, values) for val in pair]  
    instances.append(instance)  
# [...]
```

Efficacité

```
// AVERAGE_TIME.CSV  
0.000001, 0.003919, 0.000003  
0.000001, 0.003029, 0.000002  
0.000002, 0.003530, 0.000002  
0.000002, 0.004846, 0.000002  
0.000005, 0.005136, 0.000002  
0.000002, 0.006557, 0.000002  
0.000003, 0.007768, 0.000003  
0.000004, 0.007727, 0.000003  
0.000004, 0.009611, 0.000002  
0.000005, 0.007756, 0.000003  
0.000005, 0.007525, 0.000003  
0.000006, 0.010923, 0.000002  
0.000007, 0.009187, 0.000003  
0.000007, 0.010003, 0.000003  
0.000008, 0.011980, 0.000003  
0.000009, 0.009618, 0.000004  
0.000011, 0.013592, 0.000003  
0.000012, 0.015152, 0.000005  
0.000012, 0.014414, 0.000003  
0.000013, 0.015738, 0.000004  
0.000013, 0.014239, 0.000003
```

Figure 11 : average_times.csv de DP (1ère colonne), DPV (2ème) et DPW (3ème)

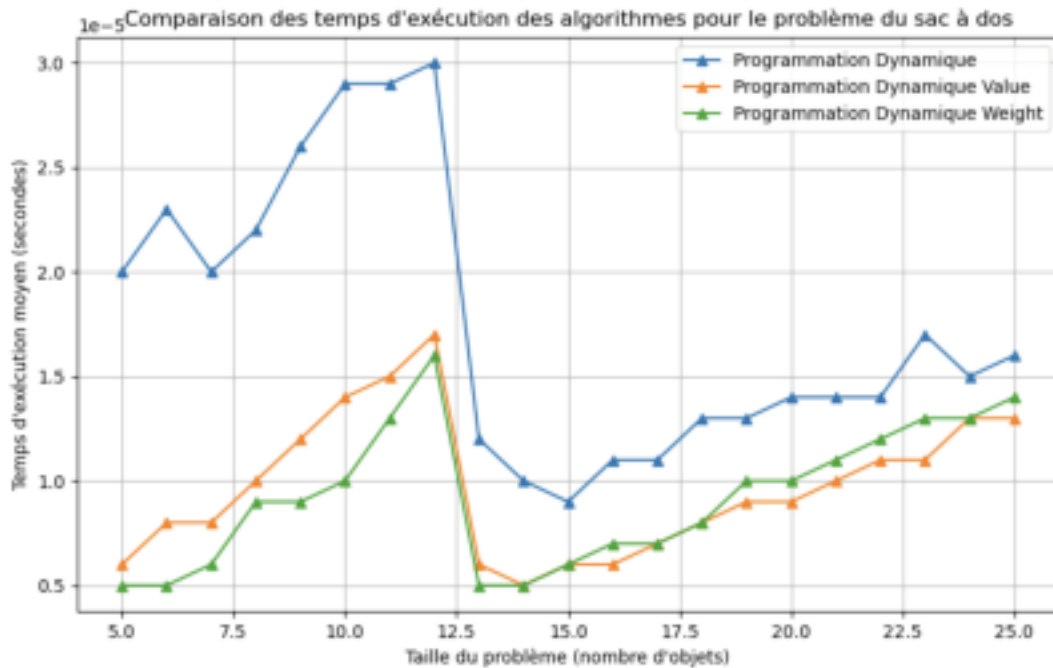


Figure 12 : Temps d'exécution (en sec) pour $n=25$, $NUM_INSTANCE_PER_SIZE=10$ et $MIN_CAPACITY=50$

Complexité temporelle

La complexité temporelle de la fonction `knapsackDP_Weight` est déterminée par les boucles imbriquées dans la fonction, (car les autres boucles sont négligeables). Dans le pire des cas, la boucle interne est parcourue C fois, et celle externe n fois. Ce qui donne une complexité temporelle de $O(nC)$.

Complexité spatiale

De même, la complexité spatiale est déterminée par notre tableau de données `dp` de n lignes et C colonnes d'où une complexité spatiale de $O(nC)$.

Question 8 :

Fonctions ajoutées :

```
int compare(const void * a, const void *b){
    Item * i = (Item *) a;
    Item * j = (Item *) b;
    if ((*i).value/(*i).weight < (*j).value/(*j).weight)
        {return 1; }
        else { return -1;}
}

int knapsack_tri(Item items[], int n, int C){
    qsort(items, (size_t) n, sizeof(int), compare);
    int v = 0;
    int w = C;
    for(int i = 0; i < n ; i++){
        if (items[i].weight <= w) {
            v += items[i].value;
            w -= items[i].weight;
        }
    }
    return v;
}
```

Exemples d'instances pour lesquelles la fonction n'est pas optimale :

Dans le tableau trié, le rapport du premier objet étant meilleur que le second, il est d'office pris. Donc si le second a une meilleur valeur mais qu'il n'y a plus assez de place pour rentrer dans le sac, il ne sera pas pris. On passe donc à côté d'une solution plus optimale. On trouve par ailleurs, plusieurs instances pour lesquelles la fonction n'est pas optimale. Parmi elles :

27, 2, 10, 18, 36, 27, 40 → le programme renvoie 36 qui n'est pas la solution optimale.

Complexité temporelle : La complexité temporelle de la fonction `knapsack_tri` est déterminée par le tri rapide (car la boucle est négligeable). Or puisque le coût d'un tri rapide est de $O(n \log(n))$, il en va de même pour notre fonction.

Complexité spatiale : La complexité temporelle de la fonction `knapsack_tri` est déterminée par la dimension du tableau passé en argument donc $O(n)$.